SAÉ S1.01 : Méthodologie d'Implémentation d'un Besoin Client et Conception Algorithmique

Thème principal : Conception logicielle, modélisation et programmation par contrat

Synthèse exécutive : Ce cours pose les fondations méthodologiques indispensables du premier projet intégrateur (SAÉ S1.01) du BUT Informatique. Il explique comment transformer un besoin client informel et ambigu en spécifications fonctionnelles rigoureuses, structurer un programme en appliquant la décomposition descendante (Top-Down), séparer strictement la logique métier des interfaces graphiques ou textuelles, et garantir la robustesse du code via la programmation par contrat, les assertions et les tests unitaires.

De l'Expression du Besoin à la Spécification Fonctionnelle

En informatique professionnelle, un projet échoue rarement à cause d'une incapacité technique à coder, mais presque toujours à cause d'un décalage entre ce que le client a exprimé, ce que le développeur a compris, et ce qui a été testé. L'objectif initial d'un projet informatique consiste donc à transformer un énoncé informel en un ensemble de règles explicites et non ambiguës. Toute imprécision lexicale (comme l'utilisation de termes vagues tels que 'rapide', 'ergonomique' ou 'gestion des erreurs') doit être bannie au profit de seuils quantifiables et de comportements déterministes.

- Expression de besoin : formulation subjective et parfois contradictoire fournie par l'utilisateur final
- Spécification fonctionnelle : traduction technique univoque décrivant le comportement attendu du système sans présager de l'implémentation
- Critères d'acceptation : conditions observables permettant d'arbitrer si une fonctionnalité est valide (triptyque 'Étant donné / Quand / Alors')

Cas concret enseignant : *L'enseignant illustre ce problème avec l'analogie classique de la 'balançoire d'entreprise' : le client demande une balançoire suspendue à une branche d'arbre, mais chaque corps de métier (chef de projet, architecte, développeur) l'interprète différemment jusqu'à couper le tronc de l'arbre en deux pour faire passer la corde, alors que le client voulait simplement un pneu attaché avec un câble.*

Décomposition Modulaire et Conception Descendante

Pour concevoir un programme robuste, l'étudiant doit proscrire la rédaction d'un bloc de code monolithique dans une seule fonction 'main'. La démarche requise est la décomposition descendante. Un problème global P est découpé en n sous-tâches P1, P2, ..., Pn. Chaque sous-tâche se matérialise par une fonction dotée d'une responsabilité unique. Il est primordial de séparer systématiquement la logique métier (calculs, algorithmes purs, tris, transformations) de la couche d'interface utilisateur (affichages écran via printf/cout/print et saisies clavier).

- Conception descendante (Top-Down) : démarche consistant à scinder un problème complexe en sous-problèmes élémentaires indépendants
- Modularité : découpage du code en modules autonomes présentant une forte cohésion interne et un faible couplage avec les autres modules
- Signature de fonction : contrat liant l'identifiant de la fonction, la nature de ses arguments d'entrée et son type de retour
- Principe de responsabilité unique (Single Responsibility Principle) : chaque fonction ne doit réaliser qu'une seule tâche précise

Cas concret enseignant : *Métaphore de la chaîne de montage automobile : le technicien qui assemble l'alternateur n'a pas besoin de savoir quelle est la couleur des sièges en cuir. De même, si une fonction de calcul fiscal contient des instructions d'affichage à l'écran, elle devient totalement inutilisable sur un serveur web ou dans une application mobile.*

Programmation par Contrat et Stratégie de Test

Une fonction mathématique ou informatique n'est fiable que si son domaine de validité est formellement délimité. La programmation par contrat stipule que si l'appelant respecte les préconditions, la fonction s'engage à satisfaire les postconditions. Les tests unitaires doivent impérativement couvrir le cas nominal (usage standard), les cas particuliers (données extrêmes, zéros, chaînes vides) et les cas d'erreur attendus.

- Précondition : hypothèse indispensable sur les paramètres d'entrée pour que la fonction garantisse un résultat correct
- Postcondition : état du système et valeur de retour garantis par la fonction une fois son exécution terminée
- Test unitaire : fonction automatisée vérifiant qu'une unité de code (fonction isolée) réagit conformément aux spécifications sur un jeu de données précis
- Assertion (assert) : instruction de contrôle stoppant immédiatement l'exécution en phase de débogage si une précondition est violée

Cas concret enseignant : *Le professeur insiste sur l'accident historique de la fusée Ariane 5 (vol 501 en 1996) : l'explosion a été provoquée par une conversion d'un nombre flottant 64 bits en entier 16 bits sans vérification des préconditions sur la plage de valeurs admissibles, entraînant un dépassement arithmétique et la perte du lanceur.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne peut-on pas mettre de 'printf' ou d'affichages dans nos fonctions de calcul d'algorithmes ?
Réponse du professeur : Parce que cela crée un couplage fort avec la console. Une fonction de calcul doit être pure et réutilisable dans n'importe quel contexte (interface graphique, application web, test automatisé sans interface).

Question étudiant : Quelle est la différence exacte entre une précondition et une condition de validation de saisie utilisateur ?
Réponse du professeur : La validation de saisie interagit avec l'humain pour lui demander de corriger son entrée en cas d'erreur. La précondition est un contrat interne entre développeurs : si elle est violée, c'est qu'il y a un bug dans le code appelant, d'où l'utilisation d'une assertion pour crasher immédiatement le programme en phase de test.

■ Exercice & Démonstration d'Application

Énoncé : Spécifier et implémenter une fonction modulaire 'calculer_moyenne_ponderee' qui prend en entrée une liste de notes (flottants sur 20) et une liste de coefficients entiers correspondants, puis renvoie la moyenne pondérée.

Démarche de résolution :
Étape 1 (Spécification et Contrats) : Définir les préconditions (taille > 0, notes entre 0 et 20, coefficients > 0) et postconditions (retourne un flottant entre 0 et 20). Étape 2 (Algorithme) : Initialiser les accumulateurs à 0. Parcourir les tableaux de 0 à taille-1 en effectuant les produits note*coefficient. Étape 3 (Cas limites) : Tester avec un tableau de taille 1 et vérifier le comportement nominal.

```
/* ------------------------------------------------------------------------ */
/* Auteur : IUT de Bayonne et du Pays Basque - Matéi de Saint Leger         */
/* Matière : S1.01 - Traitement algorithmique et modulaire                  */
/* ------------------------------------------------------------------------ */

// CONSTANTES
// (aucune constante globale nécessaire)

// VARIABLES
// (les variables locales sont gérées dans la fonction)

// TRAITEMENTS
float calculer_moyenne_ponderee(float tab_notes[], int tab_coeffs[], int taille)
{
    // (clavier/mémoire) >> validation >> préconditions
    assert(taille > 0);

    float somme_points;
    int somme_coeffs;
    int i;

    somme_points = 0.0;
    somme_coeffs = 0;
    i = 0;

    while (i < taille)
    {
        assert(tab_notes[i] >= 0.0 && tab_notes[i] <= 20.0);
        assert(tab_coeffs[i] > 0);

        somme_points = somme_points + (tab_notes[i] * tab_coeffs[i]);
        somme_coeffs = somme_coeffs + tab_coeffs[i];
        i = i + 1;
    }

    assert(somme_coeffs > 0);
    return (somme_points / somme_coeffs);
}
```

Résultat : La fonction est totalement découplée de toute entrée/sortie console. Elle garantit l'intégrité arithmétique et peut être validée instantanément par une batterie de tests unitaires automatisés.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le risque majeur d'un couplage fort entre le traitement algorithmique et l'affichage console ?

R : L'impossibilité de réutiliser la fonction dans un autre contexte (serveur, interface graphique) et l'incapacité d'automatiser les tests unitaires.

Q : Qu'appelle-t-on un cas limite (edge case) en conception algorithmique ?

R : Une situation critique aux frontières du domaine de validité (ex: tableau vide, valeur minimale ou maximale) qui provoque souvent des bugs si elle n'est pas explicitement traitée.

Q : À quoi sert une assertion (assert) dans un sous-programme ?

R : À vérifier formellement qu'une précondition est respectée pendant l'exécution et à stopper immédiatement le programme en cas d'anomalie pour faciliter le débogage.